



ARCHITECTURE GUIDE

Store Tracker: Database & Deployment Architecture

One Vercel project with Neon, versus a separate backend project with CORS. How the setup can grow with cloud services. And, for research only, how a database changes when an AI model is connected to it.

Prepared for Moriya Business · 9 October 2026

1. Summary

The Store Tracker is a web application for a local store: the cashier records stock and daily sales, and the owner sees weekly tithe and offering amounts, statistics, profit per product, and warnings. It has two halves: a **frontend** (Next.js, the pages people see) and a **backend** (FastAPI, which owns the rules and the data). The data lives in a **Neon Postgres** database. This guide explains how those pieces are connected today, how that differs from the traditional way of deploying them as two separate projects, how the setup could grow, and (as pure research) how an AI model changes the picture.

Key conclusions

1. Today the frontend and backend are two *services* inside **one** Vercel project, behind one web address. The browser only ever talks to that one address, so no CORS configuration is needed and session cookies stay simple and secure.
2. The traditional alternative, a second Vercel project for the backend plus a `CORS_ORIGIN` setting, works, but adds CORS, cross-site cookie problems, a route guard that stops working, two deployments that can drift apart, and duplicated settings.
3. Recommendation: keep the current design. Split into two projects only if the backend must serve other clients (for example a mobile app) or separate teams need separate release schedules.
4. As the business grows, add cloud services one stage at a time, driven by a real need (Section 8).
5. If an AI model is ever added, never give it direct access to the production database. Put a small, checked tool layer between them (Section 9).

How to read this guide

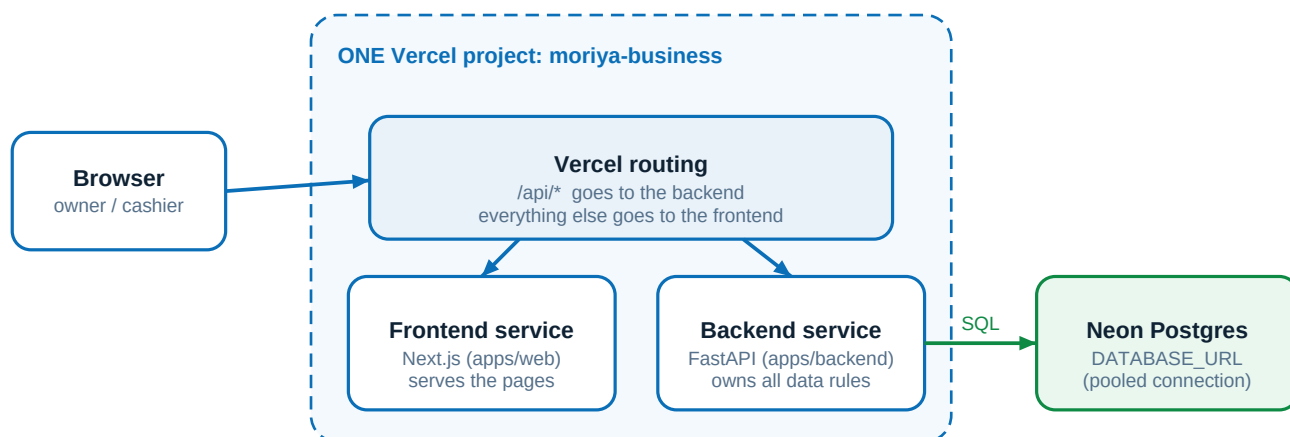
- **Section 2** defines the terms used.
- **Sections 3 to 6** compare the current way and the traditional way.
- **Section 7** covers the Neon database itself and how it is wired in.
- **Section 8** is the growth path with cloud services.
- **Section 9** is the AI research section. It is not part of the project and nothing in the application uses AI.

2. Terms used in this guide

Term	Plain-language meaning
Frontend	The pages shown in the browser (Next.js). It asks the backend for data.
Backend / API	The program (FastAPI) that checks logins, applies the business rules, and reads and writes the database.
Vercel project	One deployable thing on Vercel with its own settings, environment variables, and web address.
Service	A part of a single Vercel project that is built separately (here: frontend and backend) but deployed together.
Origin / site	The scheme + address of a page, such as <code>https://moriya-business.vercel.app</code> . Browsers apply security rules when a page talks to a different origin.
CORS	Cross-Origin Resource Sharing: the server-side permission slip that lets a page from one origin call an API on another. Without it the browser blocks the response.
Preflight	An automatic extra request (method OPTIONS) the browser sends before some cross-origin calls to ask whether they are allowed.
Cookie flags	HttpOnly (scripts cannot read it), Secure (https only), SameSite (when the browser may send it across sites).
Pooled vs direct URL	Neon offers two connection strings. Pooled goes through PgBouncer and suits app traffic. Direct suits migrations, dumps, and replication.
Migration	A versioned change to the database structure or data (we use Alembic).
Preview deployment	A temporary copy of the site for a branch, with its own URL.

3. The current implementation

Everything is one Vercel project named `moriya-business`. Its configuration file (`vercel.json`) declares two services and the rules for routing requests between them.



Everything the browser touches lives at ONE address, so cookies stay simple and no CORS is needed.

Figure 1. The current design: one project, two services, one address, Neon behind the backend.

3.1 What happens when someone logs in

- The browser loads a page from `moriya-business.vercel.app` (frontend service).
- The page calls `POST /api/auth/login` on *the same address*. Vercel routes anything under `/api/` to the backend service.
- FastAPI checks the password against the bcrypt hash in Neon and replies with a cookie named `session` (HttpOnly, Secure, SameSite=Lax).

- On the next page request the browser sends the cookie automatically. The frontend's route guard (`proxy.ts`) sees it and lets the visitor in; if it is missing the visitor is sent to the login page.
- Every page's data calls (`/api/products`, `/api/weekly`, ...) again go to the same address, carry the cookie, and are checked by the backend, which is the real security gate.

3.2 Settings the project needs

Setting	Used by	Who sets it
DATABASE_URL	Backend (pooled Neon connection)	Created automatically when the Neon integration was added
DATABASE_URL_UNPOOLED	Alembic migrations, dumps (direct connection)	Created by the same Neon integration
AUTH_SECRET	Signs the session tokens. The backend refuses to start on Vercel without a real value.	Us (random, stored as a hidden value)
CRON_SECRET	Protects the Monday weekly-report endpoint that Vercel Cron calls	Us (random, hidden)

3.3 Evidence that it behaves as described

- A browser test on the live site showed every API call going to `moriya-business.vercel.app` itself, and a product added through the page appeared in the Neon database.
- A request pretending to come from another website received **no** CORS permission headers (the API answered 405 to the preflight), so other sites cannot call the API from a browser.

3.4 Strengths and limits

Strengths	Limits
No CORS code to write or get wrong. Cookies are first-party, so browsers do not block them.	Frontend and backend always deploy together. You cannot release one without the other.
One project, one set of settings, one deployment history and one rollback.	Both services share one project's limits, logs, and plan.
Previews are consistent: a preview of the frontend always talks to the matching preview of the backend.	If another client (a mobile app) needs the API, it can call the same public <code>/api</code> address, but the API is then tied to this project.
The frontend's route guard can see the session cookie.	Services built on different runtimes still need separate dependency upkeep (Node and Python).

4. The traditional alternative: a second Vercel project plus CORS

The suggested alternative is the classic layout: deploy the backend as its own Vercel project with its own web address, give it the database settings and a `CORS_ORIGIN` value naming the frontend, and make the frontend call the backend's address directly.

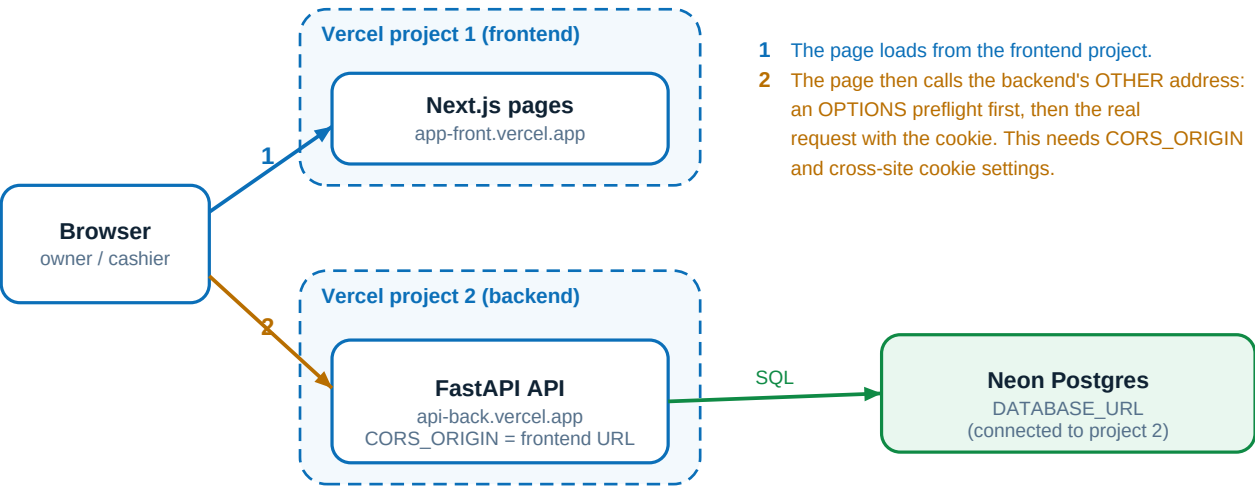


Figure 2. Two projects, two addresses. The browser treats them as different sites.

4.1 What would have to change

#	Change	Why it is needed
1	Create a second Vercel project whose root is apps/backend, with its own vercel.json (no services block).	It gets its own URL, for example api-xyz.vercel.app.
2	Connect the Neon integration to this project too, and set AUTH_SECRET, CRON_SECRET and CORS_ORIGIN there. Keep the frontend project's settings separate.	Each project has its own environment variables. Secrets now live in two places.
3	Add CORS middleware to FastAPI: allow only the frontend origin, allow credentials, list methods and headers.	Without it the browser blocks every response from the API. A wildcard (*) is not allowed when cookies are used.
4	Change the session cookie to SameSite=None; Secure, or put both projects on subdomains of one custom domain and set the cookie's Domain to the parent.	A cookie from one site is not sent along with a request from another site unless it is explicitly cross-site.
5	Give the frontend an API base URL setting and send credentials: "include" on every call.	Browsers do not send cookies on cross-origin calls by default.
6	Rewrite the frontend route guard (proxy.ts).	It reads the cookie, but the cookie now belongs to the <i>backend's</i> address, so the frontend can no longer see it.
7	Make every preview deployment of the frontend point to the matching preview of the backend, and accept that preview origin in CORS_ORIGIN.	Otherwise a preview page calls production, or is blocked.
8	Handle Vercel Deployment Protection between preview deployments (bypass tokens).	Protected previews answer a cross-project call with a login page instead of data.

4.2 What the extra code would look like (illustrative, not in the project)

```
# FastAPI: allow ONLY the frontend, and let it send the session cookie
app.add_middleware(
    CORSMiddleware,
    allow_origins=[settings.cors_origin], # exact origin, never ""
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["Content-Type"],
)

# Cookie that works across two different sites
response.set_cookie("session", token, httponly=True, secure=True, samesite="none")
```

4.3 Why cross-site cookies are the hardest part

`vercel.app` is on the Public Suffix List, which means two projects at one `vercel.app` and two `vercel.app` count as *different sites*, not as neighbours. A cookie the backend sets is then a third-party cookie, and several browsers restrict or block those by default. The login can work in one browser and silently fail in another. Two clean ways around it:

- **Custom domain for both:** `app.example.com` and `api.example.com`, with the cookie's Domain set to `.example.com`. Now they are the same site and `SameSite=Lax` works. (Needs a domain name.)
- **Rewrite proxy (a hybrid):** keep two projects but have the frontend forward `/api/*` to the backend address. The browser again sees one origin, so CORS and cookie issues disappear, at the price of one extra hop and a configuration to maintain.

Worth noticing

The fix that most teams end up choosing for two-project setups is essentially to *recreate* what the single project with services gives for free: one public address in front of both halves.

5. Old way vs new way, side by side

In this table the **traditional way** means two Vercel projects connected by CORS, and the **current way** means one project with two services and Neon attached to it.

Topic	Traditional: two projects + CORS	Current: one project, two services
Addresses the browser sees	Two (frontend and backend)	One
CORS	Required: <code>CORS_ORIGIN</code> setting and middleware; preflight requests	Not needed; the API sends no CORS permission at all
Session cookie	<code>SameSite=None</code> (third-party) or a shared custom domain	<code>SameSite=Lax</code> , first-party, <code>HttpOnly</code> , <code>Secure</code>
Browser privacy blocking	Can silently break login in some browsers	Not affected
Frontend route guard	Cannot read the backend's cookie; must be redesigned	Works unchanged
Deployments	Two, independent. Version skew is possible between them	One atomic deployment; rollback restores both together
Preview environments	Each frontend preview must be pointed at a matching backend preview and allowed in CORS	Automatically matched
Environment variables	Duplicated across two projects; secrets in two places	One project, one set
Database connection	Neon integration connected to the backend project; frontend never touches it	Neon integration on the project; only the backend service uses it
Extra latency	Preflight round trip on many calls (cacheable)	None
Attack surface	A second public origin; a mistake in CORS (for example reflecting any origin) exposes the API	Smaller; other sites cannot call the API from a browser
Independent scaling and ownership	Easy: separate teams, plans, logs, release cadence	Shared project; separate teams are harder
Other clients (mobile app, partners)	Natural: the backend is a standalone API	Possible, but the API lives inside this project
Effort to set up and keep right	Higher and ongoing	Lower; already done and tested

6. Deciding between them

Choose the current design when...	Choose two projects when...
One small team looks after both halves.	Different teams own the frontend and backend and must release independently.
The API is only used by this website.	The backend must serve other clients such as a mobile app, other websites, or partners.
You want the simplest, safest default.	The backend needs different plans, regions, or limits from the frontend.
You do not own a custom domain yet.	You have a custom domain and want <code>api.example.com</code> as a public product.

This is not a one-way door

Moving later is possible. Because all calls already go through relative `/api/...` paths, splitting into two projects later mostly means creating the second project, adding the rewrite proxy or CORS, and re-pointing the cookie settings. Nothing in the database or the business rules would change.

A middle path worth knowing: Vercel Services also allows a service to be *private* (not exposed publicly) and reached only from the other service through a binding. That would suit a future internal service, such as a background worker, without adding a public address or any CORS.

7. The Neon database and how it is wired in

7.1 From SQLite to Neon

During development the backend uses a small SQLite file (`dev.db`) so anyone can run the project with no setup. In production it uses **Neon**, a managed Postgres database. The same code runs on both: the connection address comes from the `DATABASE_URL` setting, and the app converts Neon's address style into the one the Python driver expects. Safeguards: on Vercel the backend refuses to start if the address still points at a SQLite file or if the signing secret is the development default. The same migrations were verified against a real Postgres before going live.

7.2 How the database was attached

- Neon was added from the Vercel Marketplace with one command. That created a Neon project and wrote its connection settings straight into the Vercel project's environment variables, for production, preview and development.
- Because it is attached to the project, there is no password to copy by hand and nothing to commit to Git.
- The tables and the first two accounts were created from a laptop using the *direct* (unpooled) connection, which is the right choice for schema changes. The running app uses the *pooled* connection.

7.3 What Neon provides that matters for this project

Neon feature	What it means in practice	Relevance here
Branching	Instant copy-on-write copies of the database	Test a migration on a copy of production before running it for real
Instant restore	Go back to an earlier moment; how far back depends on the plan	Recover from a bad import or accidental deletion
Autoscaling and scale-to-zero	Compute grows with load and pauses when idle; the first query after a pause is slower	Low cost for a small shop; expect a brief delay after quiet periods
Connection pooling	PgBouncer in front of the database (hostname with -pooler)	Serverless functions open many short connections; pooling keeps them safe
Read replicas	Extra read-only compute sharing the same storage	Later: heavy reports without slowing the till
IP allow lists, logical replication	Restrict who can connect; stream changes to other systems	Later: security hardening and analytics copies

Verify before relying on it

Plan limits (backup history length, storage, compute hours, branch counts) change over time. Check the current Neon plan details before relying on any specific number.

7.4 Practical rules the project follows (and one it should adopt)

- Use the **pooled** address for the app and the **direct** address for migrations and dumps. Mixing them up produces confusing errors such as missing tables or "prepared statement already exists".
- Money is stored as whole numbers of ngwee, so the database never holds fractional kwacha and sums are exact.
- Never run tests against the real database. (This was learned the hard way; the test suite now aborts if it is not pointed at its own test file.)
- **To adopt:** before each future migration, create a Neon branch of production, run the migration there, check the app, then apply it to production. So far this was not needed because production held no business data, but it will be once the shop is using the app.

7.5 A region note

The current database and the backend functions both run in the US East region. For users in Zambia that adds noticeable round-trip time on every action. When it matters, choose a region closer to the users that *both* Vercel and Neon offer, and keep the database and backend in the same region (otherwise every query pays for the distance).

8. Growing with cloud services

The application is deliberately small: one project, one database. Cloud services are useful, but every service added is another bill, another secret to protect, and another thing that can fail. The rule of thumb is to add a service only when a real problem appears, and in roughly this order.

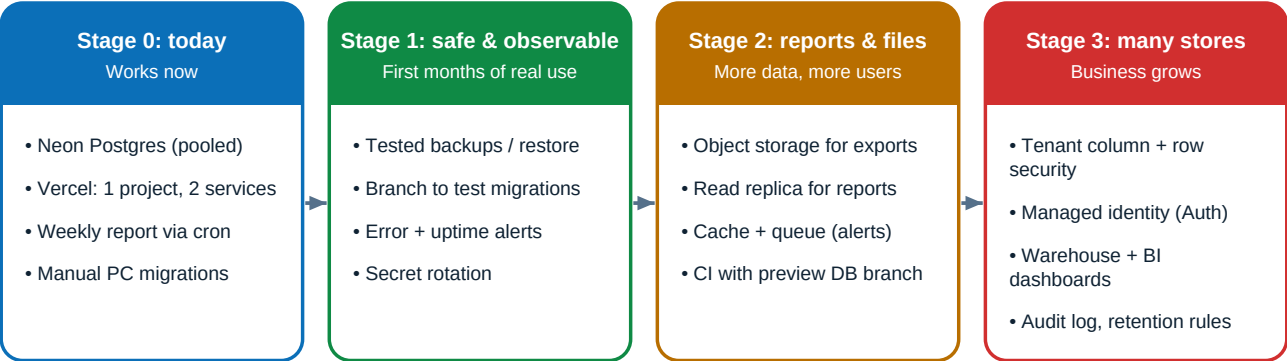


Figure 3. A growth path, from today to many stores.

8.1 Stage 1: safe and observable (first months of real use)

Need	Cloud service or practice	Why now
Do not lose data	Neon instant restore (confirm the history window), plus a scheduled export of the database to separate storage. Practise a restore at least once.	The shop's sales history is the product. A backup that has never been restored is only a hope.
Change the database safely	Neon branches to rehearse migrations; run them from CI instead of a laptop	Removes the 'works on my machine' risk
Know when it breaks	Error tracking and uptime alerts (Vercel logs and observability, or a service such as Sentry), alert if the Monday report job fails	A silent failure is worse than a loud one
Protect accounts	Owner screen to change passwords (Phase 6), login rate limiting, rotate AUTH_SECRET and CRON_SECRET	The sample passwords are public in the repository

8.2 Stage 2: reports and files (more data, more users)

Need	Cloud service or practice	Why
Monthly Excel archives	Object storage (for example Vercel Blob or an S3-compatible bucket) to keep generated reports, instead of the database. Matches the archive plan in the project plan, section 11.	Keeps the database small and cheap; files are not good database cargo
Heavy reports	A Neon read replica so statistics and exports never slow down the till	Reads and writes stop competing
Faster repeated pages	A cache (Vercel Runtime Cache or a Redis-style service) for statistics that change rarely	Stats pages recompute sums on every visit
Alerts that reach people	A queue plus an email or SMS provider for low-stock and expiry messages	Retries and delivery tracking, without making the sale screen wait
Confidence in changes	CI that runs the tests and a preview deployment with its own database branch	Catches mistakes before they reach the shop

8.3 Stage 3: many stores or staff

Need	Cloud service or practice	Why
More than one store	Add a store identifier to every table and enforce it with Postgres row-level security	One shop must never see another's data, even through a coding mistake
Real identity	A managed identity service (Neon Auth, Clerk, and similar) for password reset, MFA, and sign-in history	Replaces hand-built account handling
Business analytics	Copy changes to an analytics store (logical replication or ETL) and use a dashboard tool	Long-running analysis does not belong on the live database
Accountability	An audit log of who changed stock or edited a sale, plus data-retention rules	Planned in Phase 6; becomes essential with several staff
Network hardening	IP allow lists or private networking, secrets manager, periodic disaster-recovery drills	Lowers risk as the data becomes more valuable

8.4 What to do first

- Change the owner password and finish Phase 6 (roles, settings, audit trail).
- Confirm Neon's restore window and do one real restore test.
- Start rehearsing migrations on a Neon branch.
- Add error alerts for the API and for the Monday report job.
- Move the database and backend to a region near the users, if the delay is noticeable.

9. Research note: a database with an AI model attached

Not part of the project

This section is background research only. The Store Tracker does not use an AI model, and nothing here is planned for it. It explains how the database side of a system changes when a language model is part of the architecture.

9.1 Four common ways a model touches data

Pattern	How it works	Typical risk
A. Tools over an API (recommended)	The model can call a short list of backend functions, such as <i>get weekly summary</i> or <i>list warnings</i> . The backend checks who is asking and runs fixed queries.	Low. Capability is limited to what the tools allow.
B. Text-to-SQL	The model writes SQL from a question and the system runs it.	High on a production database: data leaks, slow queries, accidental writes.
C. Retrieval (search by meaning)	Text is turned into number vectors (embeddings) stored in the database. A question is matched to the closest stored items and those are given to the model.	Medium. Stale or over-shared content; extra storage and indexing.
D. Agents that write	The model plans several steps and changes data (create orders, edit stock).	Highest. Needs approvals, limits, and an undo path.

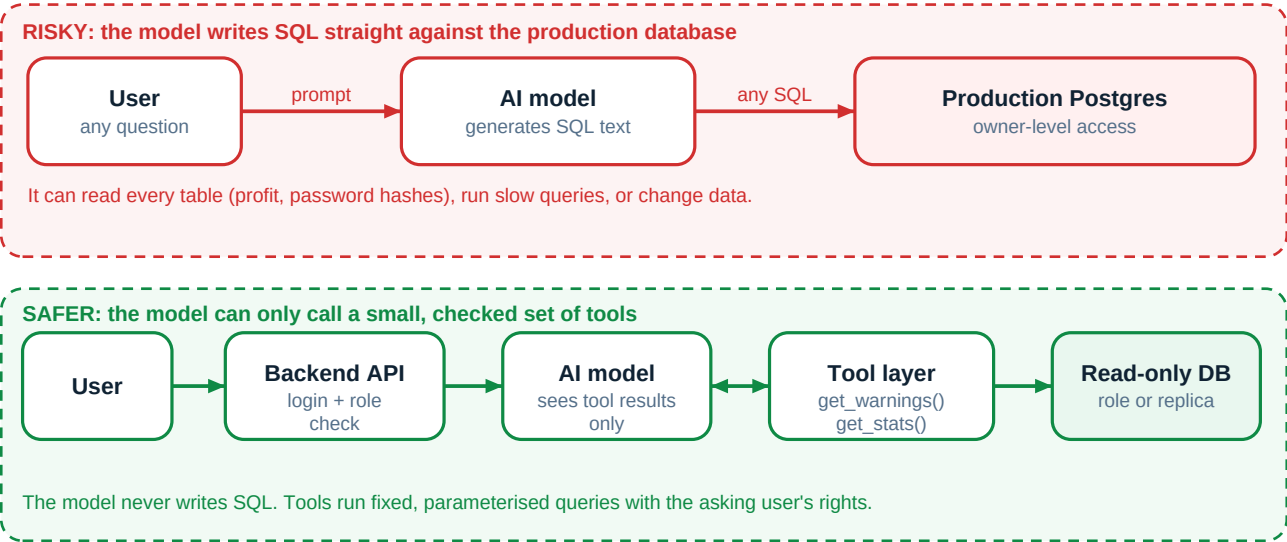


Figure 4. Giving a model raw database access, versus a small checked tool layer.

9.2 How the database side changes

Concern	Today (no model)	With a model attached
Who writes the queries?	Developers. Every query is reviewed and tested.	With text-to-SQL the model does, at run time, so queries are unpredictable. Prefer fixed, parameterised queries behind tools.
Access control	The backend uses one database login; the app checks owner vs cashier.	The model must act with <i>the asking user's</i> rights, not the app's. Use a read-only database role, restricted views, and row-level security. A cashier's question must never surface profit.
Query cost	Known queries with known cost.	A careless generated query can scan every table. Add statement timeouts, row limits, and run on a read replica.
Untrusted text	Product names and notes are only displayed.	Stored text can contain instructions (prompt injection). Anything read from the database and shown to the model is untrusted input, not a command.
Privacy	Password hashes and personal data stay in the backend.	Data sent to a model provider leaves your system. Send the minimum, never secrets or hashes, and check the provider's retention terms.
Auditability	Normal request logs.	Log who asked, what was asked, which tools ran and what data was returned, so answers can be traced.
Writes	Forms with validation and confirmation dialogs.	Model-proposed changes must pass the same validation and a human confirmation, and be idempotent (safe to retry).
Schema design	Names optimised for developers.	Clear names, comments, and 'friendly' views help a model understand data. Units matter: the app stores money in ngwee; a model could misread 1250 as K1,250 unless a view or tool returns formatted kwacha.
New data types	Numbers, dates, text.	Embeddings (vector columns) with vector indexes. They must be re-created when the source text changes and take noticeable space.
Connections	Steady, modest traffic.	Bursty: one chat can fire several tool calls at once. Use pooled connections and rate limits.
Testing	Deterministic unit tests.	Answers vary run to run. Use a fixed test set of questions, check the tool calls made (not just the wording), and re-run when prompts or models change.
Cost and speed	Database time only.	Model calls add latency and per-token cost. Pre-aggregate common answers (the stats and weekly endpoints already do) and cache them.

9.3 A worked example using this project

Question: "Which products should I restock this week?"

Approach	What happens
Safer (tools)	The backend authenticates the user, calls the existing warnings function, and returns a short structured list (product, quantity left, days of stock). The model turns that into a sentence. It never sees other tables, hashes, or profit unless the user is the owner and the tool allows it.
Risky (raw SQL)	The model is handed the schema and writes its own query on the production database with the app's login. It may join tables it should not see, ignore the owner/cashier split, or run an expensive query during opening hours.

9.4 How Neon-style features help in AI-assisted systems

- **Branching:** give an experimental agent its own copy-on-write branch of the data, so mistakes never touch production.
- **Read replicas:** isolate model-driven reads from the live workload.
- **Vector and full-text search:** Postgres can store embeddings and run similarity or keyword search, so retrieval can live beside the business data instead of in a separate system. (Check which extensions the current Neon plan offers.)
- **Pooling and autoscaling:** absorb the bursty traffic that tool-calling creates.
- **Instant restore:** a safety net if an automated change goes wrong.

9.5 A short checklist if this is ever pursued

- Start with read-only tools over the existing API, not free-form SQL.
- Use a dedicated read-only database role, and apply the same owner/cashier rules to the model.
- Treat all stored text as untrusted. Keep secrets, hashes and unneeded personal data out of every prompt.
- Add timeouts, row limits, rate limits, and complete audit logs from day one.
- Require human confirmation for any change, and rehearse agent work on a database branch.
- Measure quality with a fixed test set before and after every change of model or prompt.

10. Where things stand, and references

Item	Status
Live site	https://moriya-business.vercel.app (one project, two services)
Database	Neon Postgres via the Vercel Marketplace; pooled URL for the app, direct URL for migrations
Money	Stored as whole ngwee (1 kwacha = 100 ngwee), shown as kwacha with two decimals
Weekly report	Created every Monday 06:00 UTC by Vercel Cron, protected by CRON_SECRET; also created on demand when the Reports page is opened
Source code	github.com/amisilumbu07/Moriya_business

Further reading

- Neon documentation: branching, connection pooling, instant restore, read replicas, and search (neon.com/docs).
- Vercel documentation: Services, Cron Jobs, Marketplace integrations, and Deployment Protection (vercel.com/docs).
- MDN Web Docs: Cross-Origin Resource Sharing (CORS), and cookies (SameSite attribute).
- OWASP: guidance on prompt injection and on the top risks for applications using large language models.

Product and plan details described here were accurate when this guide was written (October 2026) but change over time. Confirm limits and prices on the providers' own pages before making decisions.

© 2026 Moriya Business. All rights reserved.